

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA CƠ KHÍ - BỘ MÔN CƠ ĐIỆN TỬ



NHẬP MÔN THỊ GIÁC MÁY TÍNH
EXERCISE 3

LỚP TN01 - HK 251

GVHD: PGS.TS. LÊ ĐỨC HẠNH

SINH VIÊN THỰC HIỆN:

STT	Họ và tên	MSSV
1	Võ Hữu Dư	2210604
2	Đào Trọng Chân	2210350
3	Dương Quang Duy	2210497
4	Trần Quang Đạo	2210647

TP.HCM, Ngày 27 tháng 11 năm 2025

Mục lục

1	Question 1	2
1.1	Tìm tọa độ, diện tích, chu vi, góc (độ) của hình sử dụng image moment và lập trình so sánh với hàm OpenCV	2
1.1.1	Tính tay sử dụng image moment	2
1.1.2	Lập trình kiểm tra và so sánh với kết quả sử dụng hàm OpenCV . .	4
1.2	Đếm số lượng xe của mỗi nhóm trên lane tương ứng nhóm từ 1 $\rightarrow$ 6 từ lane trái ngoài cùng qua phải (nhóm 5)	5

Danh sách bảng

Danh sách hình vẽ

1	Kết quả lập trình tính toán các thông số	4
2	Kết quả sử dụng hàm OpenCV để kiểm tra	5
3	Hình ảnh click 2 điểm tạo ra đường thẳng màu xanh để đếm xe đi qua . .	6
4	Hình ảnh hiển thị kết quả đếm xe tạm thời trên từng frame	8

1 Question 1

1.1 Tìm tọa độ, diện tích, chu vi, góc (độ) của hình sử dụng image moment và lập trình so sánh với hàm OpenCV

1.1.1 Tính tay sử dụng image moment

Hình có ma trận sau:

0	0	0	0	0	0	0
0	0	0	255	0	0	0
0	0	255	255	255	0	0
0	255	255	255	255	255	0
0	255	255	255	255	255	0
255	255	0	0	0	255	255
0	0	0	0	0	0	0

- Tọa độ tâm của hình và diện tích

Công thức tổng quát moment của ảnh

$$M_{p,q} = \sum_i \sum_j i^p j^q I(i,j) \quad (1)$$

Tọa độ tâm của ảnh được tính bằng công thức

$$x_c = \frac{M_{1,0}}{M_{0,0}} \quad (2)$$

$$y_c = \frac{M_{0,1}}{M_{0,0}} \quad (3)$$

Ta có

$$M_{0,0} = \sum_i \sum_j I(i,j) = 1 \times 18 = 18$$

$$\begin{aligned} M_{1,0} &= \sum_i \sum_j i \cdot I(i,j) \\ &= 1 \times (0 \times 1 + 1 \times 3 + 2 \times 3 + 3 \times 4 + 4 \times 3 + 5 \times 3 + 6 \times 1) = 54 \end{aligned}$$

$$\begin{aligned} M_{0,1} &= \sum_i \sum_j j \cdot I(i,j) \\ &= 1 \times (0 \times 0 + 1 \times 1 + 2 \times 3 + 3 \times 5 + 4 \times 5 + 5 \times 4 + 6 \times 0) = 62 \end{aligned}$$

Suy ra

$$S = M_{0,0} = 18$$

$$x_c = \frac{54}{18} = 3$$

$$y_c = \frac{62}{18} = 3.4444 \approx 3$$

0	0	0	0	0	0	0
0	0	0	b	0	0	0
0	0	b	0	b	0	0
0	c	0	0	0	c	0
0	c	c	a	c	c	0
c	c	0	0	0	c	c
0	0	0	0	0	0	0

• **Chu vi**

Contour của hình và loại pixel thể hiện trong hình
Chu vi của hình được tính bằng

$$P = n(a) + n(b)\sqrt{2} + n(c)\frac{1 + \sqrt{2}}{2}$$

Trong đó $n(a)$, $n(b)$, $n(c)$ lần lượt là số lượng pixel loại a, b, c.
Vậy ta có

$$P = 1 + 3 \times \sqrt{2} + 10 \times \frac{1 + \sqrt{2}}{2} \approx 17.3137$$

• **Góc của hình (độ)**

Góc xoay của hình được tính bằng công thức

$$\theta = \frac{1}{2} \arctan \left(\frac{2\mu'_{11}}{\mu'_{20} - \mu'_{02}} \right) \quad (4)$$

Trong đó

$$\mu'_{11} = \frac{M_{1,1}}{M_{0,0}} - x_c y_c$$

$$\mu'_{20} = \frac{M_{2,0}}{M_{0,0}} - x_c^2$$

$$\mu'_{02} = \frac{M_{0,2}}{M_{0,0}} - y_c^2$$

Lần lượt tính các moment cần thiết

$$\begin{aligned} M_{1,1} &= \sum_i \sum_j ijI(i,j) = 0 + 1 \times (3 + 4 + 5) + 2 \times (2 + 3 + 4) + 3 \times (1 + 2 + 3 + 4) \\ &\quad + 4 \times (2 + 3 + 4) + 5 \times (3 + 4 + 5) + 6 \times 5 = 186 \end{aligned}$$

$$M_{2,0} = \sum_i \sum_j i^2 I(i,j) = 0 + 1^2 \times 3 + 2^2 \times 3 + 3^2 \times 4 + 4^2 \times 3 + 5^2 \times 3 + 6^2 \times 1 = 210$$

$$M_{0,2} = \sum_i \sum_j j^2 I(i,j) = 0 + 1^2 \times 1 + 2^2 \times 3 + 3^2 \times 5 + 4^2 \times 5 + 5^2 \times 4 + 6^2 \times 0 = 238$$

Suy ra

$$\mu'_{11} = \frac{186}{18} - 3 \times \frac{62}{18} = 0$$

$$\mu'_{20} = \frac{210}{18} - 3^2 = \frac{8}{3} \approx 2.67$$

$$\mu'_{02} = \frac{238}{18} - \left(\frac{62}{18} \right)^2 = \frac{110}{18} \approx 1.358$$

Thay vào công thức 4 ta có

$$\theta = \frac{1}{2} \arctan \left(\frac{2 \times 0}{2.67 - 1.358} \right) = 0^\circ$$

• Kết luận

Các giá trị tính toán tay theo yêu cầu đề bài như sau:

- Tọa độ tâm: $(3, 3.444) \approx (3, 3)$
- Diện tích: 18 pixel
- Chu vi: ≈ 17.3137
- Góc: 0°

1.1.2 Lập trình kiểm tra và so sánh với kết quả sử dụng hàm OpenCV

Đoạn code lập trình kiểm tra kết quả tính toán

```

1  ys, xs = np.where(img == 255)
2  M00 = len(xs)
3  M10 = np.sum(xs)
4  M01 = np.sum(ys)
5  M11 = np.sum(xs * ys)
6  M20 = np.sum(xs**2)
7  M02 = np.sum(ys**2)
8  x_centroid = M10 / M00
9  y_centroid = M01 / M00
10 u11 = M11 / M00 - x_centroid * y_centroid
11 u20 = M20 / M00 - x_centroid**2
12 u02 = M02 / M00 - y_centroid**2
13 theta = 0.5 * np.arctan2(2 * u11, u20 - u02) # in radians
14 print("M00: ", M00, " M10: ", M10, " M01: ", M01)
15 print("M11 = ", M11, " M20 = ", M20, " M02 = ", M02)
16 print("Area: ", M00, " pixels")
17 print("Centroid: ", (x_centroid, y_centroid), "approx", (int(x_centroid),
18   int(y_centroid)))
19 print("Orientation angle (degrees): ", np.degrees(theta))

```

Kết quả

```

M00: 18 M10: 54 M01: 62
M11 = 186 M20 = 210 M02 = 238
Area: 18 pixels
Centroid: (np.float64(3.0), np.float64(3.4444444444444446)) ≈ (3, 3)
Orientation angle (degrees): 0.0

```

Hình 1: Kết quả lập trình tính toán các thông số

Đoạn code sử dụng hàm OpenCV để kiểm tra kết quả

```

1  contours, _ = cv2.findContours(img, cv2.RETR_LIST, cv2.CHAIN_APPROX_SIMPLE)
2  for contour in contours:
3      # Centroid
4      M = cv2.moments(contour)
5      if M["m00"] != 0:

```

```

6     cx = int(M["m10"] / M["m00"])
7     cy = int(M["m01"] / M["m00"])
8     print(f"Centroid: ({cx}, {cy})")
9
10    # Area
11    area = cv2.contourArea(contour)
12    print(f"Area: {area}")
13
14    # Perimeter
15    perimeter = cv2.arcLength(contour, True)
16    print(f"Perimeter: {perimeter}")
17
18    # Orientation
19    if len(contour) >= 5:
20        ellipse = cv2.fitEllipse(contour)
21        angle = ellipse[2]
22        print(f"Orientation angle: {angle} degrees")
    
```

Kết quả

```

Centroid: (3, 3)
Area: 10.0
Perimeter: 17.313708305358887
Orientation angle: 0.0 degrees
    
```

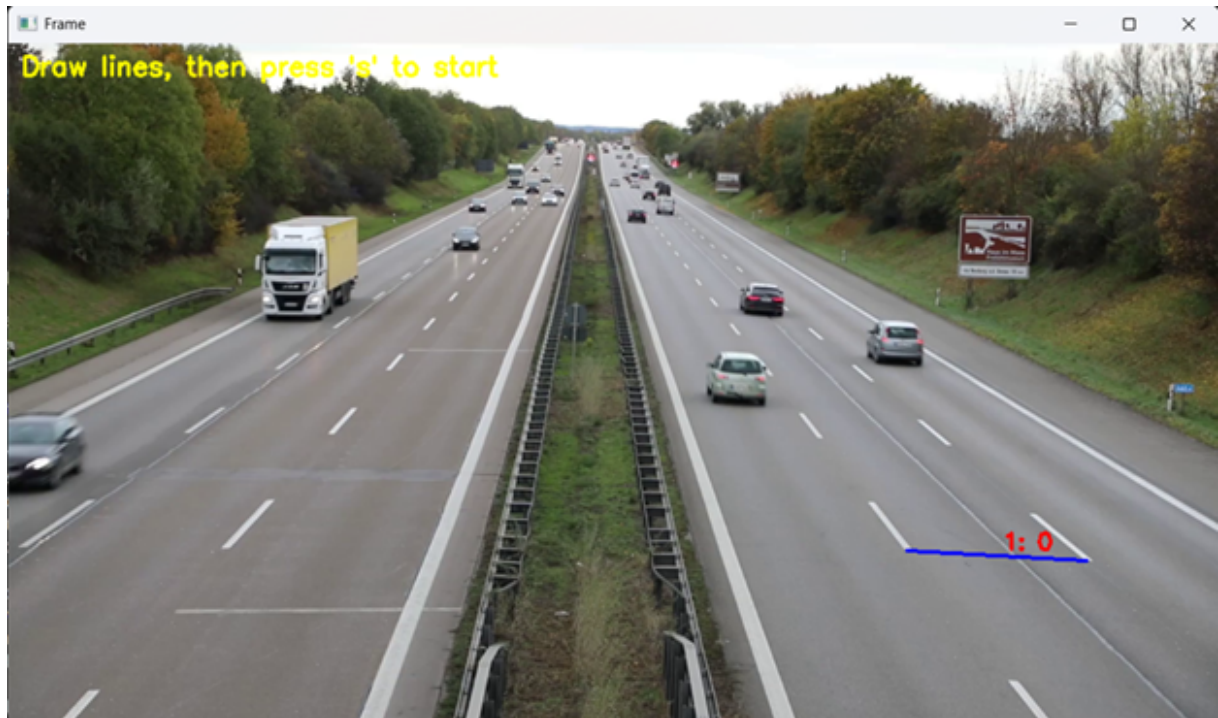
Hình 2: Kết quả sử dụng hàm OpenCV để kiểm tra

1.2 Đếm số lượng xe của mỗi nhóm trên lane tương ứng nhóm từ 1 → 6 từ lane trái ngoài cùng qua phải (nhóm 5)

Các bước thực hiện

- **Bước 1: Đọc video và vẽ đường đếm**

- Chương trình mở video và đọc frame đầu tiên.
- Frame này được resize về kích thước 960×540 pixel và hiển thị lên cửa sổ "Frame".
- Người dùng click chuột trái hai lần để tạo một đường đếm: điểm đầu (p1) và điểm cuối (p2). Có thể lặp lại thao tác để tạo tối đa 6 đường đếm tương ứng với 6 làn xe.



Hình 3: Hình ảnh click 2 điểm tạo ra đường thẳng màu xanh để đếm xe đi qua

- Mỗi đường đếm được lưu với các thông tin: tọa độ hai đầu mút, biến đếm số xe (count) và biến ghi nhận frame cuối cùng đã đếm (last_count_frame). Khi hoàn tất, người dùng nhấn phím 's' để bắt đầu xử lý video.

• Bước 2: Khởi tạo bộ tách nền và các kernel hình thái học

- Sử dụng cv2.createBackgroundSubtractorMOG2 với:

```
1 history = 450,
2 varThreshold = 17,
3 detectShadows = True
```

- Tạo kernel Open dạng ellipse kích thước (3, 3) để xóa nhiễu nhỏ.
- Tạo kernel Close dạng ellipse kích thước (6, 6) để nối các vùng foreground và lấp lỗ.

```
1 backSub = cv2.createBackgroundSubtractorMOG2(
2     history=450, varThreshold=17, detectShadows=True
3 )
4
5 kernel_open = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, KERNEL_OPEN_SIZE)
6 kernel_close = cv2.getStructuringElement(cv2.MORPH_ELLIPSE,
7     KERNEL_CLOSE_SIZE)
```

• Bước 3: Xử lý từng frame trong video

- Với mỗi frame, chương trình resize về 960×540 pixel.
- Áp dụng bộ tách nền MOG2 để thu được foreground mask.
- Dùng threshold với ngưỡng 200 để loại bỏ shadow và giữ lại vùng trắng biểu diễn thân xe.

- Áp dụng lần lượt opening, closing, dilation để tối ưu hóa dạng blob của xe.
- Sử dụng cv2.findContours để tìm các contour ngoài cùng trên mask đã xử lý.

```

1 ret, frame = cap.read()
2 if not ret:
3     break
4 frame_idx += 1
5 frame = cv2.resize(frame, (960, 540))
6 # ----- 2.1. GET FG MASK -----
7 fgMask = backSub.apply(frame)
8 _, fgMask = cv2.threshold(fgMask, 200, 255, cv2.THRESH_BINARY)
9 # ----- 2.2. MORPHOLOGY (OPEN + CLOSE) -----
10 fgMask = cv2.morphologyEx(fgMask, cv2.MORPH_OPEN,
11                             kernel_open, iterations=1)
12 fgMask = cv2.morphologyEx(fgMask, cv2.MORPH_CLOSE,
13                             kernel_close, iterations=2)
14 fgMask = cv2.dilate(fgMask, kernel_close, iterations=1)
    
```

• Bước 4: Lọc blob và xác định tâm xe

- Với mỗi contour, chương trình tính diện tích và bounding box (x, y, w, h) .
- Hàm is_valid_blob kiểm tra các điều kiện:
 - + Diện tích nằm trong $[800, 50000]$ pixel.
 - + Chiều rộng và chiều cao bounding box đều lớn hơn 15 pixel.
 - + Tỷ lệ khung hình $\frac{w}{h}$ nằm trong $[0.4, 3.0]$.
- Các contour thỏa điều kiện được coi là xe, được vẽ bounding box màu xanh lá và tâm xe là điểm giữa của bounding box.

```

1 contours, _ = cv2.findContours(fgMask, cv2.RETR_EXTERNAL,
2                                 cv2.CHAIN_APPROX_SIMPLE)
3
4 for cnt in contours:
5     if not is_valid_blob(cnt):
6         continue
7     x, y, w_box, h_box = cv2.boundingRect(cnt)
8     cx = x + w_box // 2
9     cy = y + h_box // 2
    
```

• Bước 5: Kiểm tra xe cắt qua đường đếm và cập nhật biến đếm

- Dùng hàm point_to_segment_distance để tính khoảng cách từ tâm xe tới đoạn thẳng của từng line.
- Nếu khoảng cách nhỏ hơn DIST_THRESH = 10 pixel và số frame hiện tại cách frame lần cuối đếm của line đó ít nhất MIN_FRAME_GAP = 6, bộ đếm count của line được cộng thêm 1 và last_count_frame được cập nhật.
- Nhờ cơ chế này, mỗi xe chỉ được đếm một lần khi cắt qua vùng lân cận của đường đếm.

```

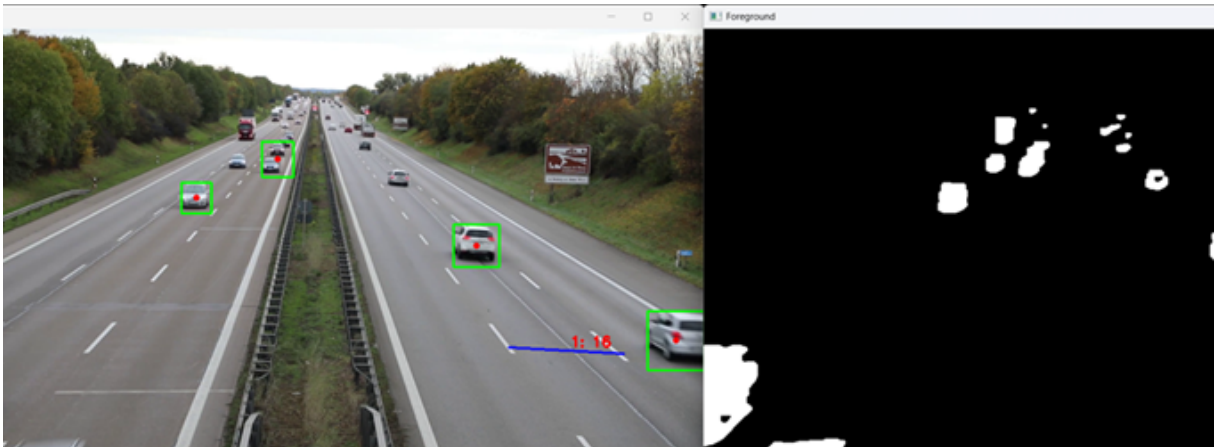
1 for line in lines:
2     x1, y1 = line['p1']
3     x2, y2 = line['p2']
4
    
```



```
5     dist = point_to_segment_distance(cx, cy, x1, y1, x2, y2)
6
7     if dist < DIST_THRESH:
8         if frame_idx - line['last_count_frame'] >= MIN_FRAME_GAP:
9             line['count'] += 1
10            line['last_count_frame'] = frame_idx
```

• Bước 6: Hiển thị kết quả tạm thời

- Trên frame màu, chương trình vẽ:
 - + Các bounding box của xe.
 - + Tâm xe bằng chấm đỏ.
 - + Các đường đếm bằng màu xanh dương và nhãn $i : n$ biểu diễn số lượng xe đã đi qua line thứ i .
 - + Cửa sổ "Foreground" hiển thị mask đen - trắng của foreground, giúp người dùng đánh giá chất lượng tách nền.



Hình 4: Hình ảnh hiển thị kết quả đếm xe tạm thời trên từng frame

• Nhận xét

Trong quá trình chạy video, khi các phương tiện đi qua vùng lân cận đường đếm trên làn số 5, tâm bounding box của chúng lần lượt cắt qua đường đếm. Với ngưỡng khoảng cách 10 pixel và khoảng cách tối thiểu giữa hai lần đếm là 6 frame, chương trình đếm được tổng cộng 16 lượt xe đi qua làn số 5. Kết quả này trùng khớp với việc quan sát thủ công trên video, do đó có thể coi là số lượng xe được đếm chính xác.